



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

DATES

Date	Section	Time	Demo Due Date	Report Due Date
10-Mar-2026	D21	2:00 to 4:50 PM	20-Mar-2026	24-Mar-2026 11:00 PM
11-Mar-2026	D31		20-Mar-2026	25-Mar-2026 11:00 PM
12-Mar-2026	D41		20-Mar-2026	26-Mar-2026 11:00 PM
13-Mar-2026	D51		20-Apr-2026	27-Mar-2026 11:00 PM

LEARNING OBJECTIVES

- To gain practical experience with the Serial Peripheral Interface (SPI) in both the controller (main) and peripheral (sub) modes.
- To gain practical experience with an SPI device, specifically the Pmod OLED screen, to delve into the fundamentals of embedded graphics programming.

HARDWARE PLATFORM AND SOFTWARE ENVIRONMENT

- The hardware platform is a Digilent Zybo Z7 development board. As with all the labs in ECE 315, CPU0 will run the FreeRTOS real-time kernel, this time with three non-idle tasks for the SPI interface.
- The fixed Zynq-7000 System-On-Chip (SoC) hardware configuration and the initial skeleton source files for Part 1 must be downloaded from the Lab 3 section of the lab canvas site. The source files contain the initial application code in C that you will be modifying to implement your designs in the two exercises.
- The AMD Pmod OLED (revision A) module will be utilized in this lab for Exercise 2, providing you an opportunity to work with graphical output via the SPI interface and to enhance your understanding of digital communication and embedded graphics programming.



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

INTRODUCTION

This laboratory exercise aims to provide students with a comprehensive understanding and hands-on experience in the utilization of the widely used Serial Peripheral Interface (SPI) communication protocol. This laboratory session is designed to facilitate an in-depth exploration of SPI in both its controller (main) and peripheral (sub) operational modes. Utilizing the advanced hardware platform provided by the Digilent Zybo Z7 development board, alongside the Pmod OLED (revision A) module, students will complete two exercises. These are intended to bridge theoretical knowledge with practical application, focusing on the manipulation of data within an SPI system and interfacing with an OLED display to generate dynamic graphical outputs. Furthermore, this laboratory seeks to empower students to apply their acquired knowledge towards the development of innovative applications or games, thereby fostering a deeper comprehension of embedded systems design and enhancing their problem-solving capabilities within a real-time operating system framework. This laboratory exercise is not only an exercise in programming but an opportunity to cultivate the skills necessary for creative engineering problem-solving and innovation in interfacing and embedded systems engineering.



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

PRE-LAB

Marks 10%

Go to the Lab 3 section in the lab Canvas site for ECE 315 and download the zipped directory for the provided project. Unzip the directory into a suitable location. Locate, open, and study the main source .c and .h source files. Separate .c and .h files are provided for Part 1 and Part 2. Note that the FreeRTOS application inside **lab3_part1.c** consists of the main function, two queues, utility variables, and the four tasks **vUartTask**, **vSpiMainTask** and **vSpiSubTask** and **vRgbLedTask** which have the following roles:

vUartTask manages the serial interface between the serial terminal on the host PC and the FreeRTOS system running on the Zybo Z7-10 CPU0. The task displays a menu of numbered commands to the user, receives command selections, and then displays the results in the terminal. This task also monitors the bytes that are sent to the serial terminal looking for a message termination sequence (defined later).

vSpiMainTask manages the SPI interface in the Zynq-7000 SoC operating in main (master) mode. The task receives data bytes from **vUartTask** through **uart_to_main_q**. If **spi_loop** is disabled, the bytes are returned directly to **vUartTask** through **main_to_uart_q** without being sent over the SPI interface.

If **spi_loopback** is enabled, **vSpiMainTask** sends the received bytes to **vSpiSubTask** through the SPI connection. This connection is implemented using three wires in the FPGA fabric that connect the **SPI0** and **SPI1** interfaces. The received bytes (from **vSpiSubTask**) are sent back to **vUartTask** through **main_to_uart_q**. Any control characters returned from the SPI tasks are removed and not displayed on the terminal.

vSpiSubTask manages the SPI1 interface in the Zynq-7000 using sub-node (slave) mode. This task exchanges data with **vSpiMainTask** over the bidirectional SPI bus. This task will be modified by you to count and return the number of bytes and messages received from **vSpiMainTask**. The number of bytes must be communicated using the following message format: "**\nNumber of bytes received over SPI:%d\nLast message byte count: %d\nTotal messages received: %d\n**". The task will send messages in this format to the serial terminal that provide the byte count and message count messages after the character bytes that were originally typed into the terminal by the user. This count message will be sent after the detection of the termination sequence '**\r%\r**' from the user and should not include those characters in the total count. Before detecting the termination sequence the device just echoes back each received byte without change.



ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

Winter 2026

As pre-lab work, construct a system architecture diagram that illustrates the data connections and control relationships among the serial terminal on the host, the UART1 interface, the SPI0 and SPI1 interfaces, the tasks, the two queues, and the global variables (loopback enable/disable variables). This diagram is also to be included in your report.



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

PART 1 - IMPLEMENTING SPI LOOPBACK

Marks 30%

Step 1: Open the provided project file in Vivado and execute the Open Block Design command in the IP INTEGRATOR submenu. Make a screenshot copy of the displayed diagram and include it in your report. Note the connections between the two SPI interfaces, SPI1 and SPI0, and explain the significance of these connections and their role in the overall design architecture.

Step 2: Synthesize the design by executing the command Generate Bitstream in the PROGRAM AND DEBUG submenu. When the synthesis steps have all completed (it will take some time), export the hardware design and ensure the “**Include bitstream**” option is selected before clicking OK. After exporting the synthesized hardware, open Vitis IDE and create a platform using the exported hardware with FreeRTOS as the operating system, make sure to modify the BSP so that **tick_rate=1000**. Next, create an application project and copy the source files provided in the Part 1 resources.

The main source code for this exercise is in **lab3_part1.c**. The files **my_uart.h** and **my_uart.c** contain the function declarations and definitions for UART communication. The files **my_spi.h** and **my_spi.c** contain the SPI read and write functions.

Step 3: Build the platform and build and run the application. Each command is selected by typing enter followed by the corresponding command number followed by Enter (<enter>command<enter>). When you run the application for the first time you will be able to change to loopback mode but no bytes will be echoed back, this functionality will be added as you finish the implementation.

Step 4: Open the source code and complete the **TODO** sections in **numerical order**. Each TODO adds additional functionality to the program, allowing you to observe the system behavior incrementally as features are implemented.

Step 5: Verify that the command “**Toggle UART Loop-back Mode**” functions correctly. The loopback feature should start off with loopback disabled. Issuing the command should enable a loopback path in **vUartManagerTask**. With the loopback enabled, text that is typed into the terminal window will be returned immediately for display in the terminal. The user can enter commands of the form ‘\r<command>\r’ to enable/disable the loopbacks, entering ‘\r%\r’ ends the text entry. Ending text entry also causes the loopback path to be disabled and the command to be ended. Information messages will be displayed in the terminal to confirm when loopback has been enabled and disabled.



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

Additionally, the RGB LED on the Zybo Z7 board will indicate the loopback state. Red will be displayed when UART loopback is enabled, providing a visual confirmation of the mode.

Step 6: Verify that the command “**Toggle SPI Loop-back Mode**” functions correctly. The loopback feature should start off with loopback disabled. Issuing the command should enable a loopback path in **vSpiMainTask**. With the loopback path disabled, text that is typed into the terminal window will be echoed back immediately. Text entry is ended by typing the termination sequence ‘\r%\r’ or ‘\r<command>\r’. Ending text entry also causes the loopback path to be disabled. Information messages will be displayed in the terminal to confirm when loopback has been enabled and disabled. Similarly, the RGB LED will be used to indicate the SPI loopback state. Blue indicates the SPI loopback is disabled and Green indicates the SPI loopback is enabled. This visual feedback will help in quickly identifying the active loopback mode.

Step 7: Verify that the default behavior in **vSpiSubTask** works correctly. When the SPI loopback is enabled, the external SPI connection is activated. However, the initial template from the Canvas will not display any output for this case. Students need to write the code to cause output to be displayed.

Step 8: You are now to modify **vSpiSubTask** so that, as well as echoing characters that are typed at the keyboard, when the message termination sequence ‘\r%\r’ is detected by this task in the stream of characters, **vSpiSubTask** will generate a string that will be sent immediately after the received bytes have been echoed over SPI back to **vSpiMainTask**. The generated string will state “\nNumber of bytes received over SPI:%d\nLast message byte count: %d\nTotal messages received: %d\n”. Where the placeholders represent the total number of bytes and the total number of messages so far that were received by **vSpiSubTask** over the SPI interface. The special control characters that are added by **vSpiMainTask** into the stream of characters sent by the user at the serial terminal will not be counted as received characters, but they will be returned to the **vSpiMainTask**. To make this work, you will also need to add the code inside **vSpiMainTask** and **vUartTask**.



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

PART 2 - SPI OLED SCREEN INTERFACE

Marks 30%

In this part of the lab, you will learn to interface with the Xilinx/AMD Pmod OLED (revision A), featuring a 128×32 pixel graphic OLED display. This module allows for complex graphical output via the SPI protocol, providing a hands-on opportunity to work with digital communication and embedded graphics programming at a high level.

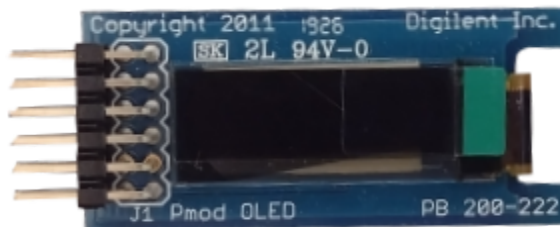


Figure 1: Pmod OLED screen

The Pmod OLED module communicates with the host board using the Serial Peripheral Interface (SPI) protocol. It requires the Chip Select (CS) line to be held low to enable data and command transmission. The state of the Data/Command (D/C) pin determines whether the transmitted bytes are interpreted as commands or data by the display controller. Specifically, when the D/C pin is pulled HIGH, the module interprets the bytes as data, allowing for the direct manipulation of pixels on the display. Conversely, when the D/C pin is pulled LOW, the bytes are transferred to the command register, enabling the execution of commands by the display controller. This flexibility allows for direct pixel manipulation, enabling the drawing of lines, shapes, and characters, as well as the rendering of bitmap images.

Pin	Signal	Description	Pin	Signal	Description
1	CS	Chip Select	7	D/C	Data/Command Control
2	MOSI	Master-Out-Slave-In	8	RES	Power Reset
3	NC	Not Connected	9	VBATC	Vbat Battery Voltage Control
4	SCK	Serial Clock	10	VDDC	Vdd Logic Voltage Control
5	GND	Power Supply Ground	11	GND	Power Supply Ground
6	VCC	Power Supply (3.3V)	12	VCC	Power Supply (3.3V)

Table 1: Pmod OLED pinout



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

To assist in application development, the manufacturer AMD/Xilinx provides the **PmodOLED_v1_0** library, which offers a suite of SPI functions and drivers for screen interfacing. However, due to performance issues with some functions, we've introduced custom replacements. The files **OLEDControllerCustom.c** and **OLEDControllerCustom.h** contain optimized versions of **OLED_DrawLineTo()**; and **OLED_RectangleTo()**; superseding the original **OLED_LineTo()**; and **OLED_DrawRect()**; While you are encouraged to familiarize yourself with the standard library functions in **PmodOLED.h**, please use the improved custom functions for these specific operations to ensure proper operation.

You are provided with a basic demo application that demonstrates the use of the Pmod OLED for drawing figures and displaying text.

Specifications:

- **Horizontal Line Sweep:** The line starts at the top of the screen and moves down to the bottom, once it reaches the bottom it starts moving up back to the top.
- **Vertical Line Sweep:** The line starts on the left side of the screen and moves to the right, when it reaches the end it changes its direction to the left.
- **Speed Variation:** The horizontal and vertical lines sweep across the screen at noticeably different speeds.

For this part of the lab you are required to develop a small application or game. This project should make use of two or more input peripherals (e.g., keypad, buttons, switches, keyboard) for user interaction, with the Pmod OLED screen serving as the output peripheral.

Requirements:

- **Display:** Your application must display both text and graphics and use a minimum of three of the available peripheral devices (pushbuttons, switches, seven segment display, keypad, RGB LED and green LEDs).
- **Input Interaction:** The input devices should influence the display in real-time. For example, you could use buttons to change displayed information (such as score or time) or to control graphical elements on the screen.
- **Creativity:** You are encouraged to be creative with your application or game design. It could be anything from a simple interactive story to a classic game adapted for the OLED display.

Assessment Criteria:

- **Functionality:** Correct implementation of the application, coding practices and functionality.



Winter 2026

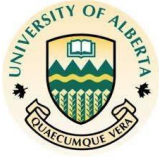
ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

- **Creativity and Complexity:** Originality of the application/game design and effective use of the OLED display and input peripherals.
- **Code Quality:** Readability, structure, and documentation of the code.

Function name	purpose
OLED_GetPos();	Fetch the current graphics drawing position.
OLED_MoveTo();	Set the current graphics drawing position.
OLED_SetCursor();	Set the character cursor position to the specified location.
OLED_PutString();	Write the specified null terminated character string to the display and advance the cursor.
OLED_DrawPixel();	Set the pixel at the current drawing location to the specified value.
OLED_RectangleTo();	Draw a rectangle bounded by the current location and the specified location.
OLED_DrawLineTo();	Draw a line from the current position to the specified position.
OLED_SetFillPattern();	Set a pointer to the current fill pattern to use.
OLED_FillRect();	Fill a rectangle bounded by the current location and the specified location.
OLED_Update();	Update the OLED display with the contents of the memory buffer.
OLED_ClearBuffer();	Clear the display memory buffer.

Table 1: Useful functions for completing Part 2



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

QUESTIONS

- How does activating the loopback mode in both (UART and SPI) impact the interaction and data flow between **vUartTask**, **vSpiMainTask** and **vSpiSubTask**?
- What difficulties did you encounter while setting up SPI communication between the main and sub devices, and what solutions did you implement?
- Describe the challenges that you faced when interfacing with the Pmod OLED screen and describe how you resolved them.
- Does input from the peripherals affect the dynamic graphics on the Pmod OLED screen? Give examples from your game.
- Reflecting on this lab's activities, how have the tasks and experiments deepened your understanding of designing embedded systems and operating within a real-time OS framework?



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

RESOURCES

- [FreeRTOS Documentation](#).
- [Zybo Z7 Reference Manual](#).
- [Pmod OLED module Reference Manual](#).



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

REPORT REQUIREMENTS

Cover Page:

- Include course and lab section numbers, lab number, and due date.
- List all student names in your lab team. Please omit ID numbers to respect privacy.

Abstract:

- Summarize the lab's objectives, the hardware used and the exercises in your own words.

Design Section:

- Outline your solutions with a concise explanation of your code for each exercise.
- Include task flow diagrams. Avoid big sections of code snapshots.

Testing Suite:

- Provide a table with test descriptions, expected and actual results and a rationale for each test case.

Conclusion:

- Assess if the given objectives were met and summarize the lab's findings.
- Discuss any issues if the lab was incomplete or unsuccessful. What were the cause(s) of the issue and how could you resolve them if you had more time.

Formatting and Submission:

- Submit the report as a single PDF file with lab number and student names in the filename.
- Ensure correct spelling and grammar.
- Use a readable font size (larger than size 10).
- Diagrams may be hand-drawn with a straightedge and scanned at 300 pixel per inch resolution.
- Source code in your report should be syntax-highlighted to enhance readability. Syntax highlighting differentiates code elements such as keywords comments and strings.
- Adopt a professional format with consistent headings and subheadings.
- Only submit your modified source code files in one zip folder. Do not zip the entire workspace.
- Adhere to the submission deadlines as indicated on the first page of the lab handout.



Winter 2026

ECE 315 Computer Interfacing

LAB 3: Zynq-7000 SPI Interface

MARKING SCHEME

Students will be graded based on the form and general quality of the report (clarity, organization, tidiness, spelling, grammar, communication effectiveness) for the lab. The Pre-lab work is worth 10%. The lab demos are worth 60% and the report is worth 30%. The demonstrations for Parts 1 and 2 are worth 30% each.